TripWiz — Folder 12: Risk Register & Top-Risk Mitigation

Team: TripWiz — Ariel University · David Kitinberg, Amit Bitton, Sagi Hassid

Risk register (Excel): 12/12-risk-register.xlsx

Contents of this folder

File	Purpose
`12/12-risk-register.xlsx`	Excel workbook with 12 project risks, custom Probability and Impact scales, and calculated **Exposure** (חשיפה)
`12.md` (this document)	Explanation of the scales and a full mitigation plan for the **highest-exposure risk**

Custom rating scales

Per submission requirements, Probability and Impact are **not** expressed as 0–1 fractions and **not** in dollars. TripWiz uses integer scales of **1–5** with explicit labels.

Probability (1–5)

Score	Label	Meaning
1	Rare	Less than once per year under normal operations
2	Unlikely	Once or twice per year; no recent incident
3	Possible	Could occur quarterly; known industry precedent
4	Likely	Expected monthly during growth or seasonal peaks
5	Almost certain	Weekly or more frequent without additional controls

Impact / Cost (1–5)

Score	Label	Meaning
1	Negligible	Minor UX inconvenience; auto-recovers within seconds
2	Low	Single user affected; workaround available without support
3	Medium	Multiple users affected; manual intervention required
4	High	Service degradation, SLA breach, or significant reputational harm
5	Critical	Full outage, data breach, regulatory exposure, or major financial loss

Exposure (חשיפה)

```
Exposure = Probability × Impact
```

Valid range: **1–25**. Higher scores indicate risks that should be mitigated first.

Risk register summary

The full register with all 12 rows is in **12/12-risk-register.xlsx** (sheet *Risk Register*). Scale definitions are on the *Scales* sheet.

Risk (short)	P	I	**Exposure**
Concurrent WebSocket edits → conflicts / lost updates	**4**	**4**	**16**
Leaked presigned S3 URLs expose trip attachments	3	5	15
Open-Meteo outage blocks weather validation	4	3	12
Bedrock usage spike exceeds budget	3	4	12
SES delivery failures hide alerts	4	3	12
Clean-account SAM deploy fails	3	4	12
PII in CloudWatch logs	3	4	12
Location Service throttling on place search	3	3	9
Compromised admin Cognito account	2	5	10
DynamoDB delete without PITR	2	5	10
WebSocket authorizer bypass	2	5	10
Lambda cold-start latency	5	2	10

Highest exposure (score 16): concurrent WebSocket edits. The mitigation plan below addresses this risk.

Highest-risk mitigation — Concurrent WebSocket edits cause itinerary conflicts or lost updates

Risk description

TripWiz supports **real-time collaborative editing** (Feature #29): multiple travelers can open the same trip and edit the itinerary simultaneously over a WebSocket connection. Each `edit` message is applied server-side and broadcast to every other connected client.

The architecture already uses **optimistic concurrency control** — every trip carries a monotonically increasing `version` field, and DynamoDB updates include `ConditionExpression: version = expectedVersion`. A conflicting write returns HTTP **409 VERSION_CONFLICT** instead of silently overwriting data.

Despite this safeguard, the risk remains **Likely (4)** with **High (4)** impact because:

1. **REST vs. WebSocket race** — A user can save via REST (`PUT /trips/{id}`) while a collaborator pushes a WebSocket `edit` on the same trip. Both paths mutate the same DynamoDB item; if the REST path does

not enforce the same version check, one channel can win without the other client being notified.
2. **Whole-document patches** — WebSocket edits can replace entire `itinerary` or `metadata` objects. Two users editing *different stops* in the same array still contend on one version counter; the second save is rejected even though the changes were logically independent.
3. **Client stale-version loops** — If the frontend does not reload the latest trip after a 409, the user may retry with an outdated `expectedVersion`, causing repeated failures and perceived data loss.
4. **No merge semantics** — There is no operational transform (OT) or CRDT layer; conflicts require manual user resolution, which is error-prone during live co-planning sessions.

Business impact: Travel partners lose trust in collaboration, re-enter data manually, or abandon shared planning — directly undermining a core differentiator of TripWiz.

Current controls (already implemented)

Control	Location	What it does
Optimistic locking	`backend/src/ws_handler/index.js` — `buildEditUpdate()`	Rejects edits when `expectedVersion` ≠ stored `version`
Version propagation	`frontend/src/pages/TripPage.jsx`, `TripOverviewPage.jsx`	`versionRef` tracks server version; WebSocket broadcasts refresh local state
Role enforcement	WebSocket `edit` handler	Only `owner` and `editor` roles may write
Conflict response	WebSocket handler `catch` block	Returns `409 VERSION_CONFLICT` with structured error code

These controls reduce severity from *Critical* to *High* but do not eliminate the likelihood during peak co-editing (pre-trip weekends, group travel planning).

Mitigation plan

1. Unify concurrency on REST and WebSocket (Priority: Immediate)

Action: Require `version` on every `PUT /trips/{tripId}` request and apply the same DynamoDB conditional write used by the WebSocket handler.

Implementation:

- Add `version` to the REST update schema; reject requests missing it with `400`.
- On `ConditionalCheckFailedException`, return `409` with `{ code: 'VERSION_CONFLICT', latestTrip }` so the client can merge.
- Log conflict rate in CloudWatch (custom metric `TripVersionConflicts`) for monitoring.

Residual risk after mitigation: Low — both write paths share one consistency model.

2. Field-level / granular patches (Priority: Short term)

Action: Extend the WebSocket `edit` payload to support **JSON Patch** (RFC 6902) operations on `itinerary` items (e.g., `replace /itinerary/3/title`) instead of replacing the full array.

Implementation:

- Server applies patch to the current itinerary server-side, still under version lock.
- Reduces false conflicts when collaborators edit different stops.

- Frontend sends minimal deltas from the trip editor component.

Residual risk after mitigation: Medium — simultaneous edits to the *same* stop still conflict (expected behavior).

3. Client-side conflict UX (Priority: Short term)

Action: When the client receives `409 VERSION_CONFLICT` (REST or WebSocket), show a non-blocking banner: *"Another traveler updated this trip. Review changes and retry."* with buttons **Reload latest** and **Keep my edits (copy to clipboard)**.

Implementation:

- On 409, fetch `GET /trips/{id}` and diff against local state (title + itinerary length + changed stop IDs).
- Highlight divergent fields in the UI before the user re-submits with the new `version`.
- Auto-reload on incoming WebSocket `edit` broadcasts (already partially implemented via `versionRef`).

Residual risk after mitigation: Low — users retain agency; data is not silently discarded.

4. Session coordination hints (Priority: Medium term)

Action: Broadcast lightweight **editing locks** over the existing `cursor` channel — e.g., `{ stopId, field, userId }` — so collaborators see who is actively editing a stop (Google Docs–style presence).

Implementation:

- No hard server lock required initially; visual indicator reduces simultaneous edits on the same field.
- Optional server-side soft lock (30 s TTL in DynamoDB) for high-contention fields if metrics show persistent conflicts.

Residual risk after mitigation: Low for typical 2–3 user sessions.

5. Monitoring & alerting (Priority: Ongoing)

Action: Define operational thresholds and alerts.

Metric	Threshold	Alert
`VERSION_CONFLICT` rate	> 5% of WebSocket edits in 5 min	SNS → ops email
WebSocket `edit` latency p99	> 2 s	CloudWatch alarm
Concurrent editors per trip	> 10	Log warning (possible abuse)

Mitigation timeline

Phase	Timeframe	Deliverables
Phase 1	Week 1–2	REST `version` enforcement, 409 response with `latestTrip`, CloudWatch metric
Phase 2	Week 3–4	JSON Patch edits, conflict banner UI
Phase 3	Week 5–6	Editing presence indicators, optional soft locks
Ongoing	Post-launch	Dashboard + SNS alerts on conflict spike

Expected residual exposure after mitigation

Dimension	Before	After (target)
Probability	4 (Likely)	2 (Unlikely)
Impact	4 (High)	3 (Medium)
Exposure	**16**	**6**

Success criteria

- Version conflict rate stays below **2%** of collaborative edit sessions in production.
- Zero user-reported incidents of *silent* itinerary data loss during co-editing in the first 90 days post-launch.
- Manual QA scenario: two browsers editing the same trip for 10 minutes — both users see consistent final state without support intervention.

How to use the Excel file

1. Open 12/12-risk-register.xlsx in Microsoft Excel or Google Sheets.
2. Review the **Scales** sheet for Probability, Impact, and Exposure definitions.
3. On **Risk Register**, sort column **Exposure** descending to reprioritize as the project evolves.
4. Adjust scores when controls from this document are implemented.